

Universidade Federal de Ouro Preto - UFOP
Instituto de Ciências Exatas e Biológicas - ICEB
Departamento de Computação - DECOM
Ciência da Computação

Implementação TAD para problema do Caixeiro Viajante

BCC202 - Estrutura de Dados I

Lourrane Lindsay Alves Evaristo
Matheus Mota Gomes

Professor: Pedro Henrique Lopes Silva

Ouro Preto
4 de novembro de 2023

Sumário

1	Introdução	1
1.1	Especificações do problema	1
1.2	Considerações iniciais	1
1.3	Ferramentas utilizadas	1
1.4	Especificações da máquina	1
1.5	Instruções de compilação e execução	2
2	Desenvolvimento	3
2.1	Implementação	3
2.1.1	TAD grafoponderado	3
2.1.2	Main	7
2.2	Testes	8
2.2.1	Teste Terminal	8
2.2.2	Teste Valgrind	8
2.2.3	Teste Python	9
2.3	Análise	9
3	Conclusão	10

Lista de Figuras

1	Teste no terminal do arquivo de entrada 1.in	8
2	Teste no valgrind do arquivo de entrada 1.in	8
3	Teste no valgrind do arquivo de entrada 1.in	9

Lista de Códigos Fonte

1	Struct da TAD	3
2	Função de alocação	3
3	Função de Desalocação	4
4	Função de leitura	4
5	Função de verificação	5
6	Função de encontrar	6
7	Função de impressão	6
8	main.c	7

1 Introdução

Os tipos abstratos de dados, TAD's, se tratam de programas em linguagem de programação C que podem ser divididos em vários arquivos fonte ".c" e ".h", ou seja, são arquivos de texto com funções que representam parte da implementação que são separados em módulos. Os principais elementos de um TAD inclui a definição do nome do tipo de dados criado (struct), definindo um novo tipo, junto aos protótipos de funções para a sua criação e manipulação dentro de um arquivo ".h".

Esse tipo de estruturação serve para organizar os códigos já que facilita a manutenção do programa.

A recursividade é quando uma função é usada em função de si mesma, ou seja, quando uma função chama a si mesma. A recursão pode acontecer de forma direta, quando a chamada está contida nela, ou indiretamente, quando a função chama uma segunda função, e essa segunda função, por sua vez, retorna a primeira função que a chamou.

Mais adiante iremos apresentar o processo de desenvolvimento de um programa implementado em módulos e o uso de função recursiva para resolver o problema do caixeiro viajante.

1.1 Especificações do problema

Temos um cenário onde um caixeiro viajante necessita visitar varias cidades distintas, começando em uma cidade e retornando a ela ao final, sendo as distâncias entre as cidade variadas. Deve-se achar o caminho com a menor distância entre as cidades e para isso será utilizado função recursiva para achar o resultado.

1.2 Considerações iniciais

Algumas ferramentas foram utilizadas durante a criação deste projeto:

- Ambiente de desenvolvimento do código fonte: Visual Studio Code. ¹
- Linguagem utilizada: C.
- Ambiente de desenvolvimento da documentação: Overleaf L^AT_EX. ²

1.3 Ferramentas utilizadas

Algumas ferramentas foram utilizadas para testar a implementação, como:

- *Valgrind*: ferramentas de análise dinâmica do código.

1.4 Especificações da máquina

A máquina onde o desenvolvimento e os testes foram realizados possui a seguinte configuração:

- Processador: 1)i3-core 5005u / 2) i5-core 4210u .
- Memória RAM: 1) 04Gb / 2) 08Gb.
- Sistema Operacional: 1)Windows 10 pro / 2) Linux Ubuntu 22.04.3 LTS.

¹Visual Studio Code está disponível em <https://www.visualstudio.com>

²Disponível em <https://www.overleaf.com/>

1.5 Instruções de compilação e execução

Para a compilação do projeto, basta digitar:

Compilando o projeto

```
gcc -c grafo.c -Wall  
gcc -c main.c -Wall  
gcc grafo.o main.o -o exe -lm
```

Usou-se para a compilação as seguintes opções:

- *-Wall*: para mostrar todos os possível *warnings* do código.

Para a execução do programa basta digitar:

```
./exe caminho_até_o_arquivo_de_entrada
```

2 Desenvolvimento

O uso de TAD para programação em C especifica um conjunto de dados e operações a serem executadas com os dados criados, o que exige manipulações com ponteiros. É um método de abstração necessário quando os tipos simples da linguagem não são suficientes para representar a complexidade de um problema, necessitando da criação de novos tipos de dados.

Para a solução do problema foi feito a implementação de um programa em linguagem C onde, dado um arquivo de entrada com o número de cidades e as distâncias entre elas, deverá ser obtido uma saída mostrando o trajeto e o valor total do menor caminho encontrado.

2.1 Implementação

Foi feito a implementação de um TAD para um Grafo ponderado, dentro deste TAD. Utilizamos seis funções onde temos uma função para alocação, uma para desalocação, uma função de leitura do arquivo de entrada, função de impressão do resultado, função para encontrar a menor distância e achar o caminho recursivamente.

2.1.1 TAD grafoponderado

A estrutura do grafo ficou definida com um vetor de vetores de inteiros para armazenar os valores das distâncias entre as cidades, um vetor de inteiros para o caminho, duas variáveis do tipo inteiro para armazenar a distância e a quantidade de cidades.

Código 1 apresenta o código da Struct da TAD grafoponderado.

```
1 typedef struct grafo{
2     int **mapa; // Matriz com as distancias
3     int *caminho; // vetor do caminho
4     int ncity; // numeros de cidade
5     int distancia; // distanica do caminho
6
7 }Grafo;
```

Código 1: Struct da TAD

Função de Alocação: A função de alocação está declarada no **.h**, onde ela retorna um ponteiro para Grafo. Ela inicia alocando a struct de Grafo e fazendo a verificação se há espaço na memória. Após, faz a alocação tanto do vetor de vetores como do vetor do caminho, sempre verificando se há espaço. Inicializa as variáveis de quantidade de cidades e a distância com zero, retornando um ponteiro para uma variável do tipo Grafo.

Código 2 apresenta o código da função de alocação.

```
1 // .h
2 // Função de alocação do grafo
3 Grafo* alocaGrafo(int n);
4
5 // .c
6 // Função de alocação do labirinto com mensagem de erros em caso de erro
7 Grafo* alocaGrafo(int n){
8
9     Grafo* newgrafo = (Grafo*)malloc(sizeof(Grafo));
10    if (newgrafo == NULL){
11        printf("Memoria insuficiente.\n");
12        exit(1);
13    }
14
15    newgrafo->mapa = (int**)malloc(n * sizeof(int*));
```

```

16     if (newgrafo->mapa == NULL){
17         printf("Memoria insuficiente.\n");
18         exit(1);
19     }
20
21     for (int i = 0; i < n; i++){
22         newgrafo->mapa[i] = (int*)malloc(n * sizeof(int));
23         if (newgrafo->mapa[i] == NULL){
24             printf("Memoria insuficiente.\n");
25             exit(1);
26         }
27     }
28
29     newgrafo->caminho = (int*)malloc(n+1 * sizeof(int));
30     newgrafo->distancia = 0;
31     newgrafo->ncity = 0;
32     return newgrafo;
33 }

```

Código 2: Função de alocação

Função de Desalocação: A função de desalocação do TAD também está prototipada no `.h`, sendo ela do tipo `void`. Inicia-se fazendo a liberação da memória do vetor de caminho, após se tem a liberação de toda a matriz com as distâncias e ao final a liberação do TAD da memória.

Código 3 apresenta o código da função de Desalocação.

```

1 //.h
2 //Função de desalocação do grafo
3 void desalocaGrafo(Grafo** );
4
5 //.c
6 //Função de desalocação do labirinto
7 void desalocaGrafo(Grafo** pGrafo){
8
9     free((*pGrafo)->caminho);
10    for (int i = 0; i < (*pGrafo)->ncity; i++){
11
12        free((*pGrafo)->mapa[i]);
13    }
14    free((*pGrafo)->mapa);
15    free(*pGrafo);
16
17 }

```

Código 3: Função de Desalocação

Função de leitura: A função de leitura possui outras cinco variáveis do tipo inteiro declarada dentro dela, sendo uma para a leitura da quantidade de cidades, a 'm' para fazer o controle do loop e as outras três para pegar os valores das distâncias entre as cidades.

Código 4 apresenta o código da função de Leitura.

```

1 //.h
2 //Função de entrada de dados
3 Grafo* leGrafo(Grafo* );
4
5 //.c

```

```

6 //Leitura dos dados do labirinto
7 Grafo* leGrafo(Grafo* pGrafo){
8
9     int n,i,j,k,m;
10    i=0;
11    scanf("%d",&n);
12
13    pGrafo = alocaGrafo(n);
14    pGrafo->ncity = n;
15    while (m < (n*n)){
16        scanf("%d %d %d",&i,&j,&k);
17        pGrafo->mapa[i][j] = k;
18        m++;
19    }
20    return pGrafo;
21 }

```

Código 4: Função de leitura

Funções de Verificação e Encontrar o caminho: A função de verificação se trata de uma função onde se pega o valor da distância da cidade até a cidade inicial, faz o bloqueio de todas as cidade já visitadas. Se verifica se a cidade já foi visita e faz a busca pela menor distância presente entre as cidades não visitadas. No caso de todas já terem sido visitadas faz com que o caixeiro viajante volte a cidade inicial. Sempre ao final faz a soma da distância.

A Função de encontrar o caminho é uma função recursiva que tem como caso base o caixeiro viajante já ter visitado todas as cidades, colocando o retorno à cidade inicial e retornando que foi passível encontrar o caminho. Caso ele não tenha visitado todas as cidades se faz a adição da cidade onde ele está no vetor do caminho, faz a chamada da função de verificação para saber qual a próxima cidade e finaliza fazendo a chama recursiva para a próxima cidade.

Código 5 apresenta o código da função de Verificação.

```

1 //.h
2 //Fun o de achar o menor
3 int verificamenor(Grafo* ,int ,int* );
4
5 //.c
6 //Fun o verifica menor distancia
7 int verificamenor(Grafo *pG,int cidade,int *n){
8     int aux,proxcidade,j,inicio,aux1;
9     aux1 = 0;
10    aux = 9999999;
11    inicio = pG->mapa[cidade][0];
12    //tranca os lugares que ja passou
13    for(int i = 0;i < *n+1;i++){
14        j = pG->caminho[i];
15        pG->mapa[cidade][j] = -1;
16    }
17    //procura a menor distancia
18    for (int i = 0; i < pG->ncity; i++){
19        if(pG->mapa[cidade][i] > -1){
20            if(pG->mapa[cidade][i] < aux){
21                aux = pG->mapa[cidade][i];
22                proxcidade = i;
23                aux1 = 1;
24            }
25        }
26    }
27    //caso final de retornar a cidade inicial

```

```

28     if (aux1 == 0){
29         aux = inicio;
30         proxcidade = 0;
31     }
32
33     pG->distancia = pG->distancia + aux;
34     return proxcidade;
35 }

```

Código 5: Função de verificação

Código 6 apresenta o código da função de Encontrar o caminho.

```

1  //.h
2  //Função de encontra caminho
3  int encontraCaminho(Grafo* ,int ,int ,int );
4
5  //.c
6  //Função recursiva
7  int encontraCaminho(Grafo* pGrafo, int city,int aux,int achou){
8
9      if(aux >= pGrafo->ncity){
10         pGrafo->caminho[aux] = city;
11         return 1;
12     }else if(aux < pGrafo->ncity){
13
14         pGrafo->caminho[aux] = city;
15         city = verificamenor(pGrafo,city,&aux);
16         achou = encontraCaminho(pGrafo,city,aux+1,achou);
17     }
18
19     return achou;
20
21 }

```

Código 6: Função de encontrar

Função de impressão: A função de impressão faz a impressão da ordem das cidades visitadas e a distancia total percorrida.

Código 7 apresenta o código da função de impressão.

```

1  //.h
2  //Imprime caminho
3  void imprimeCaminho(Grafo* );
4
5  //.c
6  //Impressão da saída de acordo com a opção de entrada
7  void imprimeCaminho(Grafo* pGrafo){
8
9      for(int i = 0;i < pGrafo->ncity+1;i++){
10         printf("%d ",pGrafo->caminho[i]);
11     }
12     printf("\n%d\n",pGrafo->distancia);
13
14 }

```

Código 7: Função de impressão

2.1.2 Main

O main do programa ficou implementado da seguinte forma, inicia com a inclusão do **grafoponderado.h** e da biblioteca **stdio.h**. Vem a declaração de um ponteiro para Grafo inicializado com **NULL**, após vem a variável que irá verificar se foi possível encontrar o caminho sendo está inicializada com o valor 0. Tem a chamada da Função de leitura e a de encontrar o caminho após, faz a validação da , em caso verdadeiro imprime que não foi possível encontrar o resultado, se for falso houve resolução e faz a impressão do resultado. Ao final é feita a desalocação e se encerra o programa.

Código 8 apresenta o main do programa.

```
1 #include "grafoponderado.h"
2 #include <stdio.h>
3
4 int main(){
5
6     Grafo *pGrafo = NULL;
7     int resultado = 0;
8     pGrafo = leGrafo(pGrafo);
9     resultado = encontraCaminho(pGrafo,0,0,0);
10    if(resultado == 0){
11        printf("N o achou caminho");
12    }else{
13
14        imprimeCaminho(pGrafo);
15    }
16    desalocaGrafo(&pGrafo);
17    return 0;
18 }
```

Código 8: main.c

2.2 Testes

Para este trabalho foram feitos três testes, sendo um pelo terminal, outro pelo valgrind e um pelo corretor python das aulas praticas.

2.2.1 Teste Terminal

Entradas: arquivo de entrada 1.in. Comando `./exe <Tests/1.in` e `./exe`

No arquivo temos na primeira linha a quantidade de cidades existente e nas linhas subsequentes os valores das distâncias de cada cidade à outra.

Figura 1 demonstra o resultado tanto pela entrada manual como pelo arquivo.

```
mota@mota-H61MLV2:~/Downloads/TpEd-masters$ ./exe <Tests/1.in
8 1 3 2 0
80
mota@mota-H61MLV2:~/Downloads/TpEd-masters$ ^C

mota@mota-H61MLV2:~/Downloads/TpEd-masters$ ./exe
4
0 0 0
0 1 10
0 2 15
0 3 20
1 0 10
1 1 0
1 2 35
1 3 25
2 0 15
2 1 35
2 2 0
2 3 30
3 0 20
3 1 25
3 2 30
3 3 0
0 1 3 2 0
80
mota@mota-H61MLV2:~/Downloads/TpEd-masters$ |
```

Figura 1: Teste no terminal do arquivo de entrada 1.in

2.2.2 Teste Valgrind

Entradas: arquivo de entrada 1.in. Linha de comando: `valgrind --leak-check=full ./exe <Tests/1.in`

No arquivo temo na primeira linha a quantidade de cidades existente e nas linhas subsequentes os valores das distancias de cada cidade a outra.

Figura 2 mostra o resultado da depuração através do valgrind.

```
mota@mota-H61MLV2:~/Downloads/TpEd-masters$ valgrind --leak-check=full ./exe <Tests/1.in
==3503== Memcheck, a memory error detector
==3503== Copyright (C) 2002-2017, and GNU GPL'd, by Julian Seward et al.
==3503== Using Valgrind-3.13.0 and LibVEX; rerun with -h for copyright info
==3503== Command: ./exe
==3503==
8 1 3 2 0
80
==3503==
==3503== HEAP SUMMARY:
==3503==    in use at exit: 0 bytes in 0 blocks
==3503==   total heap usage: 9 allocs, 9 frees, 5,236 bytes allocated
==3503==
==3503== All heap blocks were freed -- no leaks are possible
==3503==
==3503== For counts of detected and suppressed errors, rerun with: -v
==3503== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)
mota@mota-H61MLV2:~/Downloads/TpEd-masters$ |
```

Figura 2: Teste no valgrind do arquivo de entrada 1.in

2.2.3 Teste Python

Entradas: arquivo de entrada corretor.py. Linha de comando: python3 corretor.py

Este teste ele ira executar o arquivo de correção, que ira verificar todos os arquivos de teste e retornar o resultado.

Figura 3 mostra o resultado no terminal.



```
mota@mota-H61MLV2:~/Downloads/TpEd-masters$ python3 corretor.py
Analisando atividade:
1.in OK
2.in OK
3.in ER
4.in ER
5.in ER
Nota na atividade: 4.00
mota@mota-H61MLV2:~/Downloads/TpEd-masters$ |
```

Figura 3: Teste no valgrind do arquivo de entrada 1.in

2.3 Analise

Com os resultados que obtivemos, podemos afirmar que a abordagem do problema no início se parece complicado, onde ao final se demonstrou não sendo. Com a tratativa do problema se tornando até simples. Não se pode negar que o uso de TAD deixa a implementação mais organizada, já que divide o programa em setores bem definidos e funcionais o que, conseqüentemente, auxilia no entendimento do funcionamento do código. Outra vantagem consiste na ocultação da implementação das funções do sistema caso isso seja do interesse do desenvolvedor.

3 Conclusão

No problema do caixeiro viajante existem muitas variáveis a serem analisadas, o número de cidades, a distancia entre as cidades, o fato de uma cidade não poder ser visitada mais de uma vez, a menos que seja a cidade inicial. Enfim muitos fatores que devem ser analisados para gerar o melhor resultado na escolha de qual caminho mais curto seguir.

O TAD vem justamente representar o agrupamento de conceitos desses conceitos mais complexos. Como o uso de TAD exige alocar e liberar memória, problemas como memory leaks podem surgir se não forem implementados de maneira adequada, o que exige boas práticas de manipulação que podem ser auxiliadas com ferramentas de depuração como Valgrind para identificar os erros e para que possam ser corrigidos. Já a utilização da função de recursão deve ser cuidadosamente definida, para que não gere um loop infinito na execução.

Com isso é fundamental definir uma condição de parada, denominado caso base, que delimite o uso da função recursiva. A recursão é indicada em casos mais complexos que exigem o uso de uma pilha, ou empilhamento, de execução, o que é o caso do problema do caixeiro viajante, já que precisamos verificar a menor distancia entre as cidades, ao mesmo tempo que sua escolha depende de uma condição inicial onde não se pode escolher uma cidade já visitada anteriormente, o que exige uma verificação periódica. Esse empilhamento gera mais consumo de memória, por isso é necessário analisar se um versão iterativa é mais viável para a resolução do problema.